Nama : Nindya Alif Romland

NIM : H1D024031

Shift Awal : A

Shift Akhir : F

TUGAS SISTEM PAKAR 1 PERTEMUAN 4

1. **Import Library**

```
import tkinter as tk
from tkinter import messagebox
```

- tkinter adalah library bawaan Python untuk membuat tampilan GUI (Graphical User Interface). Dengan tkinter ini, kita bisa membuat elemen visual seperti jendela, tombol, label teks, input field, dan lainnya tanpa perlu install library tambahan.
- from tkinter import messagebox mengimpor modul khusus untuk menampilkan kotak dialog pop-up. Modul ini punya beberapa fungsi seperti showinfo() untuk menampilkan informasi, showerror() untuk error, dan askyesno() untuk konfirmasi. Pada program ini yang dipakai adalah showinfo() untuk menampilkan hasil diagnosa.

2. **Knowledge Base**

```
# Data Kerusakan & Gejala
database_kerusakan = {
    "VGA Overheat": ["fps_drop", "laptop_panas", "kipas_berisik"],
    "RAM Kurang": ["game_lag", "stuttering", "loading_lama"],
    "Driver VGA Bermasalah": ["game_crash", "layar_blank", "artefak"],
    "CPU Bottleneck": ["fps_drop", "cpu_100", "game_tidak_stabil"],
    "Storage Lambat": ["loading_lama", "texture_lambat", "game_stuck"]
}
```

Knowledge base menyimpan seluruh pengetahuan yang dimiliki oleh sistem, dalam hal ini berupa aturan hubungan antara kerusakan dan gejala-gejalanya. Struktur data yang digunakan adalah dictionary Python. Dictionary bekerja dengan pasangan key-value:

- Key (kunci) = nama kerusakan, misalnya "VGA Overheat"
- Value (nilai) = list berisi kode-kode gejala yang berhubungan dengan kerusakan tersebut

Contohnya, "VGA Overheat" diasosiasikan dengan tiga gejala yaitu fps_drop, laptop_panas, dan kipas_berisik. Artinya menurut basis pengetahuan ini, jika ketiga gejala tersebut muncul bersamaan, kemungkinan besar masalahnya adalah VGA yang mengalami overheat.

Penggunaan dictionary dipilih karena lebih terstruktur dan mudah dikembangkan dibanding if-else panjang untuk menambah kerusakan baru cukup tambah satu baris tanpa mengubah logika program.

3. Database Solusi

```
solusi_kerusakan = {
    "VGA Overheat": "Bersihkan kipas dan gunakan cooling pad",
    "RAM Kurang": "Upgrade RAM minimal 8GB atau 16GB",
    "Driver VGA Bermasalah": "Update atau install ulang driver VGA",
    "CPU Bottleneck": "Turunkan setting game atau upgrade CPU",
    "Storage Lambat": "Gunakan SSD untuk performa lebih cepat"
}
```

Dictionary kedua ini menyimpan solusi untuk setiap kerusakan. Dipisah dari database_kerusakan agar lebih rapi dan mengikuti prinsip pemisahan data-data gejala di satu tempat, data solusi di tempat lain. Key-nya sama persis dengan database_kerusakan supaya bisa diakses dengan cara solusi_kerusakan[nama_kerusakan].

4. Daftar Semua Gejala

```
# Daftar Gejala
semua_gejala = [
    ("fps_drop", "Apakah game mengalami FPS drop?"),
    ("laptop_panas", "Apakah laptop terasa sangat panas saat bermain?"),
    ("kipas_berisik", "Apakah kipas berbunyi keras?"),
    ("game_lag", "Apakah game terasa lag?"),
    ("stuttering", "Apakah game tersendat-sendat (stuttering)?"),
    ("loading_lama", "Apakah loading game lama?"),
    ("game_crash", "Apakah game sering crash/keluar sendiri?"),
    ("layar_blank", "Apakah layar tiba-tiba blank saat bermain?"),
    ("artefak", "Apakah muncul garis atau glitch di layar?"),
    ("cpu_100", "Apakah penggunaan CPU selalu 100%?"),
    ("game_tidak_stabil", "Apakah performa game tidak stabil?"),
    ("texture_lambat", "Apakah tekstur game muncul terlambat?"),
    ("game_stuck", "Apakah game sering stuck/freeze?")
]
```

Ini adalah daftar semua gejala yang akan ditanyakan satu per satu kepada pengguna. Struktur datanya adalah list of tuple, sebuah list yang setiap elemennya berupa tuple berisi dua nilai:

- Nilai pertama [0] = kode gejala, digunakan secara internal oleh program untuk dicocokkan dengan knowledge base.
- Nilai kedua [1] = teks pertanyaan dalam bahasa Indonesia, yang ditampilkan ke layar untuk dibaca pengguna.

Alasan menggunakan list (bukan set atau dict) adalah karena list mempertahankan urutan dan bisa diakses menggunakan index angka seperti semua_gejala[0], semua_gejala[1], dan seterusnya. Ini penting karena program menampilkan pertanyaan secara berurutan menggunakan counter index_pertanyaan.

5. Class SistemPakar dan Constructor __init__

```python
# Class Aplikasi
class SistemPakar:
    def __init__(self, root):
        self.root = root
        self.root.title("Sistem Pakar Diagnosa Gaming Laptop")
        self.root.geometry("450x300")

        self.gejala_terpilih = []
        self.index_pertanyaan = 0
```

class SistemPakar mendefinisikan sebuah kelas yang menjadi cetak biru (blueprint) dari aplikasi. Semua tampilan dan logika program dikemas di dalam kelas ini menggunakan pendekatan OOP (Object-Oriented Programming).

__init__ adalah constructor, method khusus yang otomatis dipanggil pertama kali saat objek dari kelas ini dibuat. Parameter root adalah jendela utama tkinter yang dikirim dari luar saat objek dibuat.

- self.root.title(...) mengatur judul yang muncul di title bar jendela
- self.root.geometry("450x300") mengatur ukuran jendela, 450 piksel lebar dan 300 piksel tinggi
- self.gejala_terpilih = [] adalah list kosong yang nantinya akan diisi kode gejala yang dijawab "YA" oleh pengguna
- self.index_pertanyaan = 0 adalah counter yang melacak pertanyaan ke berapa yang sedang aktif ditampilkan

6. Pembuatan Widget di Constructor

```python
# Label
self.label_tanya = tk.Label(root, text="Selamat Datang di Sistem Pakar", font=("Arial", 12))
self.label_tanya.pack(pady=20)
```

tk.Label membuat widget teks statis di jendela. Parameter text adalah teks awal yang ditampilkan, dan font mengatur jenis dan ukuran huruf. .pack(pady=20) menempatkan label ke jendela dengan jarak vertikal 20 piksel di atas dan bawah. Label ini nantinya akan terus diperbarui isinya menggunakan .config(text=...) untuk menampilkan pertanyaan yang berbeda-beda.

```python
# Tombol mulai
self.btn_mulai = tk.Button(root, text="Mulai Diagnosa", command=self.mulai)
self.btn_mulai.pack(pady=10)
```

- tk.Button membuat tombol.
- Parameter command=self.mulai menghubungkan tombol ini dengan method mulai(), artinya setiap kali tombol diklik, fungsi mulai() akan otomatis dijalankan.

```python
# Frame jawaban
self.frame_jawab = tk.Frame(root)

self.btn_ya = tk.Button(self.frame_jawab, text="YA", width=10, command=lambda: self.jawab("y"))
self.btn_tidak = tk.Button(self.frame_jawab, text="TIDAK", width=10, command=lambda: self.jawab("t"))

self.btn_ya.pack(side=tk.LEFT, padx=10)
self.btn_tidak.pack(side=tk.LEFT, padx=10)
```

- tk.Frame adalah container tak terlihat yang digunakan untuk mengelompokkan widget. Di sini dipakai untuk menampung tombol YA dan TIDAK bersama-sama agar bisa ditampilkan dan disembunyikan sekaligus.
- lambda: self.jawab("y") dan lambda: self.jawab("t") digunakan karena command pada tkinter tidak bisa langsung menerima fungsi dengan argumen. Lambda membungkus pemanggilan fungsi sehingga argumen "y" atau "t" bisa diteruskan saat tombol diklik.
- side=tk.LEFT membuat kedua tombol tersusun secara horizontal (berjejer ke kiri).

7. **Method mulai()**

```python
# Mulai
def mulai(self):
    self.gejala_terpilih = []
    self.index_pertanyaan = 0

    self.btn_mulai.pack_forget()
    self.frame_jawab.pack(pady=20)

    self.tampilkan_pertanyaan()
```

Method ini dijalankan saat tombol "Mulai Diagnosa" diklik. Fungsinya adalah mempersiapkan sesi diagnosa baru dari awal.

- self.gejala_terpilih = [] mengosongkan list gejala agar diagnosa dimulai dari bersih, bukan melanjutkan sesi sebelumnya.
- self.index_pertanyaan = 0 mereset counter pertanyaan ke nol.
- .pack_forget() menyembunyikan tombol "Mulai Diagnosa" dari layar supaya tidak mengganggu saat sesi pertanyaan berlangsung.
- self.frame_jawab.pack(pady=20) menampilkan frame berisi tombol YA dan TIDAK.
- self.tampilkan_pertanyaan() langsung memanggil pertanyaan pertama.

8. **Method tampilkan_pertanyaan()**

```python
# Tampilkan Pertanyaan
def tampilkan_pertanyaan(self):
    if self.index_pertanyaan < len(semua_gejala):
        kode, teks = semua_gejala[self.index_pertanyaan]
        self.label_tanya.config(text=teks)
    else:
        self.proses_hasil()
```

Method ini bertanggung jawab menampilkan pertanyaan yang sesuai dengan index saat ini.

- Kondisi if self.index_pertanyaan < len(semua_gejala) mengecek apakah masih ada pertanyaan yang belum ditampilkan. len(semua_gejala) menghasilkan 13 (jumlah total gejala), jadi selama index masih di bawah 13, pertanyaan terus ditampilkan.
- kode, teks = semua_gejala[self.index_pertanyaan] adalah tuple unpacking, mengambil elemen tuple sekaligus dan langsung memecahnya menjadi dua variabel terpisah. kode berisi kode gejala dan teks berisi kalimat pertanyaan.
- self.label_tanya.config(text=teks) memperbarui teks label di layar dengan pertanyaan baru. Ini lebih efisien daripada membuat label baru setiap kali.
- Ketika semua pertanyaan sudah habis (index >= 13), program masuk ke blok else dan memanggil proses_hasil() untuk menghitung diagnosa.

9. Method jawab()

```
# Jawaban User
def jawab(self, respon):
    if respon == "y":
        kode = semua_gejala[self.index_pertanyaan][0]
        self.gejala_terpilih.append(kode)

    self.index_pertanyaan += 1
    self.tampilkan_pertanyaan()
```

Method ini dipanggil setiap kali pengguna menekan tombol YA atau TIDAK.

- Jika respon == "y" (tombol YA ditekan), program mengambil kode gejala dari index saat ini menggunakan semua_gejala[self.index_pertanyaan][0], angka [0] mengambil elemen pertama dari tuple, yaitu kode gejala. Kode itu kemudian ditambahkan ke self.gejala_terpilih menggunakan .append().
- Jika respon == "t" (tombol TIDAK), tidak ada yang ditambahkan, pertanyaan cukup dilewati.
- Setelah itu self.index_pertanyaan += 1 memajukan counter ke pertanyaan berikutnya, dan tampilkan_pertanyaan() dipanggil lagi untuk menampilkan pertanyaan selanjutnya. Ini menciptakan alur berulang (loop) yang berjalan sampai semua pertanyaan selesai.

10. Method proses_hasil()

```
# Proses Hasil
def proses_hasil(self):
    skor_tertinggi = -1
    hasil = ""
    solusi = ""
```

- skor_tertinggi = -1 diinisialisasi dengan nilai -1 bukan 0, supaya kerusakan pertama yang ditemukan pasti langsung menggantikannya meskipun skornya 0 sekalipun .
- hasil = "" variabel penampung nama kerusakan yang akan dipilih sebagai hasil akhir.
- solusi = "" variabel penampung teks solusi yang sesuai dengan kerusakan terpilih.

```
for kerusakan, gejala in database_kerusakan.items():
    skor = 0
    for g in gejala:
        if g in self.gejala_terpilih:
            skor += 1

    if skor > skor_tertinggi:
        skor_tertinggi = skor
        hasil = kerusakan
        solusi = solusi_kerusakan[kerusakan]
```

- database_kerusakan.items() mengambil semua pasangan key-value dari dictionary, sehingga setiap iterasi mendapat nama kerusakan dan list gejalanya sekaligus.
- skor = 0 direset ke nol setiap kali masuk ke kerusakan baru, agar perhitungan tiap kerusakan tidak saling tercampur.
- for g in gejala mengiterasi setiap gejala yang ada pada kerusakan yang sedang dievaluasi.
- if g in self.gejala_terpilih mengecek apakah gejala tersebut termasuk dalam gejala yang dijawab YA oleh user.
- skor += 1 menambah skor sebesar 1 setiap kali ditemukan gejala yang cocok.
- if skor > skor_tertinggi membandingkan skor kerusakan saat ini dengan skor tertinggi yang sudah tersimpan sebelumnya.
- Jika lebih tinggi, maka skor_tertinggi, hasil, dan solusi diperbarui dengan data kerusakan yang baru.

```
# Hitung persentase keyakinan
total_gejala = len(database_kerusakan[hasil])
persentase = (skor_tertinggi / total_gejala) * 100
```

- len(database_kerusakan[hasil]) menghitung jumlah total gejala yang dimiliki oleh kerusakan terpilih menurut knowledge base.
- Persentase dihitung dengan rumus (gejala cocok / total gejala) × 100, misalnya kerusakan "VGA Overheat" punya 3 gejala dan user cocok 2, maka hasilnya (2/3) × 100 = 67%.

```
messagebox.showinfo(
    "Hasil Diagnosa",
    f"Kerusakan yang paling mungkin:\n\n{hasil}"
    f"\n\nTingkat keyakinan: {persentase:.0f}%"
    f"\n\nSolusi:\n{solusi}"
)
```

- messagebox.showinfo() menampilkan hasil dalam bentuk kotak dialog pop-up dengan judul "Hasil Diagnosa".
- f"..." adalah f-string yang memungkinkan variabel disisipkan langsung ke dalam teks.
- \n berfungsi sebagai baris baru agar tampilan hasil lebih rapi.
- {persentase:.0f} memformat angka desimal menjadi bilangan bulat tanpa koma, misalnya 66.666... menjadi 67.

```
# Reset
self.frame_jawab.pack_forget()
self.btn_mulai.pack(pady=10)
self.label_tanya.config(text="Diagnosa selesai. Ingin mencoba lagi?")
```

- self.frame_jawab.pack_forget() menyembunyikan frame berisi tombol YA dan TIDAK dari layar setelah diagnosa selesai.
- self.btn_mulai.pack(pady=10) memunculkan kembali tombol "Mulai Diagnosa" agar user bisa mengulang sesi dari awal.
- self.label_tanya.config(text=...) memperbarui teks label menjadi kalimat ajakan untuk mencoba lagi, menandakan satu sesi diagnosa telah selesai.

11. Main Program

```
# Main Program
if __name__ == "__main__":
    root = tk.Tk()
    app = SistemPakar(root)
    root.mainloop()
```

- if __name__ == "__main__" adalah kondisi yang memastikan blok kode ini hanya dijalankan ketika file dieksekusi langsung oleh Python, bukan ketika diimport sebagai modul oleh file lain.
- root = tk.Tk() membuat objek jendela utama aplikasi, objek root inilah yang menjadi wadah dari semua widget yang akan ditampilkan.

- app = SistemPakar(root) membuat objek dari class SistemPakar sekaligus mengirimkan root sebagai argumen — pada saat inilah __init__ dijalankan dan semua widget seperti label, tombol, dan frame dibuat.
- root.mainloop() menjalankan event loop tkinter, yaitu sebuah loop tak terbatas yang terus berjalan menunggu interaksi dari pengguna seperti klik tombol atau menutup jendela — tanpa baris ini jendela akan langsung tertutup begitu dibuka, dan loop ini baru berhenti ketika pengguna menutup jendela aplikasi.